



Informe Técnico: Visualizador Avanzado de Algoritmos de Ordenamiento

Desarrollo de Software y Estructuras de Datos

Autor:

Apaza Calizaya Rodrigo Josseph
Huaylla Benito Nestor Fernando
Pachari Cahui Alessandro Ivan

Fecha:

22 de abril de 2026

Resumen: Implementación de una plataforma web interactiva para la visualización en tiempo real de la organización de pedidos mediante el algoritmo Bubble Sort y variantes de optimización. [cite: 3, 4]

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
2.1. Entorno de Despliegue y Repositorio	2
2.2. Capa de Datos (Modelo)	2
2.3. Capa de Visualización (Vista)	2
2.4. Capa de Control (Lógica)	3
3. Análisis del Algoritmo de Ordenamiento	3
3.1. Lógica de Comparación e Intercambio	3
3.2. Optimización de la Experiencia de Usuario	3
4. Resultados y Observaciones Visuales	4
5. Análisis de Rendimiento (Estadísticas)	4
6. Conclusiones	4
7. Referencias Bibliográficas	5

1. Introducción

El estudio de los algoritmos de ordenamiento constituye uno de los pilares fundamentales de la ciencia de la computación. No obstante, la comprensión abstracta de su complejidad temporal y espacial suele presentar retos para los estudiantes. El presente informe detalla la creación de un simulador visual desarrollado en HTML5, CSS3 y JavaScript, diseñado para cerrar la brecha entre la teoría y la práctica.

La herramienta permite al usuario manipular variables críticas como el tamaño de la muestra, la velocidad de ejecución y el criterio de ordenamiento (Producción, Entrega o Prioridad), proporcionando una retroalimentación visual inmediata a través de gráficos de barras dinámicos y tablas de datos sincronizadas.

2. Arquitectura del Sistema

El software se divide en tres capas funcionales que trabajan de manera asíncrona:

2.1. Entorno de Despliegue y Repositorio

El proyecto ha sido desplegado utilizando *GitHub Pages*, lo que permite el acceso público a la herramienta interactiva. El código fuente completo, junto con la estructura de archivos y recursos multimedia, se encuentra alojado en el siguiente repositorio:

<https://alessapac74.github.io/ordenament/>

2.2. Capa de Datos (Modelo)

Se utiliza el paradigma de Programación Orientada a Objetos (POO) para definir la entidad `Pedido`. Esta estructura permite almacenar múltiples atributos por cada unidad de datos, simulando un entorno logístico real.

- **tiempo_produccion:** Variable cuantitativa para la escala de altura del gráfico.
- **tiempo_entrega:** Criterio secundario de ordenamiento.
- **prioridad:** Factor de ponderación para la importancia del pedido.

2.3. Capa de Visualización (Vista)

El sistema emplea un **canvas** flexible (basado en **flexbox**) donde cada barra representa un objeto de la clase `Pedido`. Los colores de las barras se calculan dinámicamente mediante

el modelo HSL (*Hue, Saturation, Lightness*), permitiendo identificar el valor de los datos por su tonalidad.

2.4. Capa de Control (Lógica)

El motor del sistema reside en funciones asíncronas (`async/await`). Esta elección técnica es fundamental, ya que JavaScript es, por naturaleza, monohilo y no bloqueante. Sin el uso de `Promises` y funciones `sleep`, la animación de ordenamiento sería instantánea e imperceptible para el ojo humano.

3. Análisis del Algoritmo de Ordenamiento

El algoritmo central implementado es el **Bubble Sort** (Ordenamiento de Burbuja). Aunque es un algoritmo de complejidad cuadrática $O(n^2)$, su naturaleza iterativa lo hace ideal para la enseñanza de la lógica de comparación e intercambio.

3.1. Lógica de Comparación e Intercambio

En el código implementado, el algoritmo no solo reubica los elementos, sino que notifica a la interfaz qué índices están bajo examen.

```
1  async function bubble(key, asc) {
2    for (let i = 0; i < n; i++) {
3      for (let j = 0; j < n - i - 1; j++) {
4        await checkPause(); // Control de estado
5        dibujar(key, {compare: [j, j+1]}); // Feedback visual
6        if (comparar(pedidos[j], pedidos[j+1], key, asc)) {
7          [pedidos[j], pedidos[j+1]] = [pedidos[j+1], pedidos[j]];
8          intercambios++;
9        }
10     }
11   }
12 }
```

3.2. Optimización de la Experiencia de Usuario

A diferencia de implementaciones básicas, este código incluye:

1. **Control de Pausa y Detención:** Mediante una función de comprobación de estado (`checkPause`) y banderas booleanas (`pausado`, `detenido`), el usuario puede analizar un estado específico del arreglo sin interrumpir el ciclo de vida del programa.

2. **Gestión de Velocidad Dinámica:** Se implementó una fórmula inversa para el *delay*: $500 - \text{valor del slider}$. Esto garantiza que, a mayor valor en el slider, menor sea el tiempo de espera, resultando en una ejecución más rápida.

4. Resultados y Observaciones Visuales

Durante las pruebas de ejecución, se identificaron tres estados críticos que el simulador resalta mediante cambios de color en el DOM (Document Object Model):

- **Comparación (Ámbar):** Identifica los dos elementos que están siendo evaluados por la condicional de orden.
- **Intercambio (Rojo):** Se activa cuando la condición se cumple y los objetos permutan su posición en el arreglo.
- **Ordenado (Azul):** Al finalizar cada iteración externa, el elemento más grande (en orden ascendente) queda fijado en su posición final, siendo marcado para evitar comparaciones innecesarias en el siguiente ciclo.

5. Análisis de Rendimiento (Estadísticas)

El simulador incorpora un panel de estadísticas en tiempo real que contabiliza las comparaciones e intercambios.

- **Peor caso (Arreglo invertido):** Para n elementos, el número de comparaciones tiende a $\frac{n(n-1)}{2}$. El simulador permite verificar empíricamente cómo estas cifras crecen exponencialmente al aumentar el tamaño de la muestra con el slider de "Tamaño".
- **Mejor caso (Arreglo ordenado):** Si se selecciona la opción de generar datos ".ordenados", el usuario puede observar que el número de intercambios es cero, validando la teoría del algoritmo.

6. Conclusiones

La implementación de este simulador demuestra que el desarrollo web moderno ofrece herramientas potentes para la educación técnica. Se cumplieron satisfactoriamente los objetivos de la Práctica 3[cite: 3, 4], logrando no solo el ordenamiento funcional de los pedidos, sino una representación gráfica precisa de la lógica interna de los algoritmos de intercambio.

El uso de `async/await` y la manipulación dinámica del DOM resultaron ser estrategias eficaces para representar procesos de computación secuencial en un entorno asíncrono. Esta herramienta es escalable y permite, en futuras versiones, la integración de algoritmos más complejos como *QuickSort* o *MergeSort* bajo el mismo marco de visualización.

7. Referencias Bibliográficas

1. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 3: Sorting and Searching*.
2. MDN Web Docs. *Concurrency model and the event loop*.
3. Guía de Práctica 3: Ordenamiento de Pedidos[cite: 1, 180].